✅ 1. What is Ocelot API Gateway?

Ocelot is a **lightweight API Gateway** built specifically for **.NET Core microservices**.
It acts as a **reverse proxy**, **request router**, and **unified entry point** for all downstream microservices.

🧠 2. Why do we need an API Gateway in Microservices?

In microservices, we have many small services:

- Product Service
- Order Service
- Inventory Service
- Payment Service
- Identity Service
- Notification Service

If clients directly call each service:

- Many URLs → Hard to manage
- Authentication repeated everywhere
- Cross-cutting concerns duplicated (logging, rate limit, caching)
- Versioning becomes complex

👉 Solution → API Gateway

It provides:

✓ Single Entry Point

Client calls **one endpoint** → Gateway forwards request.

✓ Routing

Maps external routes to internal services.

✓ Authentication & Authorization

JWT validation done at gateway.

✓ **Aggregation**

One request → multiple downstream calls.

✓ **Load Balancing**

If a service has multiple instances.

✓ **Service Discovery (Consul, Eureka)**

Ocelot automatically finds service addresses.

🎯 **3. How Ocelot Works (Simple Diagram)**

Client → Ocelot Gateway → Product Microservice

→ Order Microservice

→ Payment Microservice

→ Inventory Microservice

Ocelot sits in front of all services and **reverse proxies** requests.

⚙️ **4. Install Ocelot in ASP.NET Core**

Step 1: Create Ocelot Gateway Project

dotnet new webapi -n APIGateway

Step 2: Install Ocelot NuGet Package

dotnet add package Ocelot

Step 3: Add ocelot.json

Create at project root:

```
{
  "Routes": [
    {
      "DownstreamPathTemplate": "/api/products",
      "DownstreamScheme": "http",
```

```
    "DownstreamHostAndPorts": [
      {
        "Host": "localhost",
        "Port": 7001
      }
    ],
    "UpstreamPathTemplate": "/product-service",
    "UpstreamHttpMethod": [ "GET" ]
  }
 ],
 "GlobalConfiguration": { }
}
```

Step 4: Register Ocelot in Program.cs

```
using Ocelot.DependencyInjection;
using Ocelot.Middleware;

var builder = WebApplication.CreateBuilder(args);
builder.Configuration.AddJsonFile("ocelot.json", optional: false, reloadOnChange: true);

builder.Services.AddOcelot();

var app = builder.Build();
await app.UseOcelot();

app.Run();
```

🚀 5. Running Microservices with Ocelot

Let's say your services run on:

Service	Port
Product Service	7001
Order Service	7002
Payment Service	7003

Update ocelot.json:

```
{
  "Routes": [
    {
      "UpstreamPathTemplate": "/products",
      "DownstreamPathTemplate": "/api/products",
      "DownstreamHostAndPorts": [
        { "Host": "localhost", "Port": 7001 }
      ],
      "DownstreamScheme": "http"
    },
    {
      "UpstreamPathTemplate": "/orders",
      "DownstreamPathTemplate": "/api/orders",
      "DownstreamHostAndPorts": [
        { "Host": "localhost", "Port": 7002 }
      ],
      "DownstreamScheme": "http"
    }
```

```
  ]
}
```

Now the client calls:

GET http://localhost:5000/products

Ocelot internally calls:

GET http://localhost:7001/api/products

🔐 6. Add JWT Authentication to Ocelot

Install package:

```
dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer
```

Add JWT config in Program.cs:

```
builder.Services
  .AddAuthentication("Bearer")
  .AddJwtBearer("Bearer", options =>
  {
     options.Authority = "https://localhost:6001"; // Identity Service
     options.Audience = "gateway_api";
  });

app.UseAuthentication();
app.UseAuthorization();
```

Add to route:

```
"AuthenticationOptions": {
  "AuthenticationProviderKey": "Bearer",
  "AllowedScopes": []
}
```

Now Ocelot handles **JWT validation** before forwarding requests.

🎯 7. Using Ocelot With Consul Service Discovery

Why Consul?

Consul allows Ocelot to find:

- Live services
- New service instances
- Automatically update routes

👉 No need to hardcode ports in ocelot.json.

Step 1: Install package

dotnet add package Ocelot.Provider.Consul

Step 2: Configure ocelot.json

```
{
  "Routes": [
    {
      "UpstreamPathTemplate": "/products",
      "UseServiceDiscovery": true,
      "DownstreamPathTemplate": "/api/products",
      "ServiceName": "product-service",
      "LoadBalancerOptions": {
        "Type": "RoundRobin"
      }
    }
  ],
```

```
  "GlobalConfiguration": {
    "ServiceDiscoveryProvider": {
      "Host": "localhost",
      "Port": 8500,
      "Type": "Consul"
    }
  }
}
```

Step 3: Register microservice in Consul

Each service Program.cs:

```
builder.Services.AddSingleton<IConsulClient, ConsulClient>(p =>
    new ConsulClient(consulConfig =>
    {
        consulConfig.Address = new Uri("http://localhost:8500");
    })
);
```

Register on startup:

```
app.Lifetime.ApplicationStarted.Register(() =>
{
    consulClient.Agent.ServiceRegister(new AgentServiceRegistration()
    {
        ID = "product-service-1",
        Name = "product-service",
        Address = "localhost",
        Port = 7001
```

```
    });
});
```

🚀 8. Running Traffic With Consul

Client → Ocelot → Consul → Picks service instance → Calls microservice

Advantages:

- Auto load balancing
- Auto discovery
- No static ports
- Dynamic scaling (add/remove instances)

📦 9. Folder Structure (Recommended)

```
/MicroservicesProject
  /APIGateway
     ocelot.json
  /Services
     /ProductService
     /OrderService
     /PaymentService
     /InventoryService
     /IdentityService
  /Consul
```

✓ 10. When to Use Ocelot

Use Ocelot if:

- Using **.NET microservices**

- Need **JWT authentication**
- Need **Service discovery**
- Need **reverse proxy**
- Using **Docker + Kubernetes**

🎉 **Final Summary**

Ocelot API Gateway provides:

✓ **Routing**

✓ **Authentication (JWT)**

✓ **Service Discovery (Consul)**

✓ **Load balancing**

✓ **Aggregation**

✓ **Reverse Proxy**

✓ **Single entry point**